

# Project 1: River Crossing Puzzle

**Starting state: 3, 3, 0, 0, L**

Quick note on how we set this up. We wrote the puzzle as a search problem with a start state, a goal test, and a successor function that returns the next state, the action, and the step cost. DFS and BFS share one graph search function, and UCS and A\* use the same loop with a priority queue instead. The only real difference between them is the fringe: DFS uses a stack, BFS uses a queue, UCS uses a priority queue ordered by  $g(n)$ , and A\* uses a priority queue ordered by  $f(n) = g(n) + h(n)$ . Every version keeps a closed set so the same state never gets expanded twice, and the goal test happens when a node comes off the fringe, not when it goes on. That last part matters for UCS and A\*, because a goal can get added with an expensive path while a cheaper one is still sitting in the queue. We count one expansion each time a state goes into the closed set and its successors get generated, so the goal never counts. Our moves always come out in the order (0,1), (0,2), (1,0), (1,1), (2,0). .

One thing we noticed while testing that explains a lot: this puzzle is really small. Only 16 states can be reached, and every solution that doesn't repeat a state is exactly 11 moves. There are only four of them. Under Cost Model A they cost 23, 25, 25, and 27. Under Model B they all cost 17.

## Question 1: DFS and BFS

DFS got 11 moves and 12 expansions. BFS got 11 moves and 14 expansions. Cost is just the number of moves here, so both are 11. They found the same path: 2 cannibals over, 1 back, 2 cannibals over, 1 back, 2 missionaries over, 1 missionary and 1 cannibal back, 2 missionaries over, 1 cannibal back, 2 cannibals over, 1 back, then the last 2 cannibals over.

Since both algorithms run the exact same code in my implementation and only get handed a different fringe, the data structure is provably the only difference between them. BFS uses a queue, so it finishes every node at one depth before going deeper. That's why it always finds a shortest path when every move costs the same. The first goal it pulls off the queue can't have a shorter version hiding behind it, because anything shorter would have come off already. DFS uses a stack and just dives down one branch, so it returns the first goal it bumps into. No guarantee at all. The tradeoff is memory. BFS has to hold a whole level at once, DFS only holds the branch it's on.

DFS did less work here and still got an optimal path, but we don't think that says anything good about DFS. It's the puzzle, not the algorithm. Since every solution without a repeated state is 11 moves, DFS can't return a worse one even if it wants to. Make the puzzle bigger and DFS would wander off while BFS still came back with a shortest path.

On efficiency versus completeness, BFS expanded 14 of the 16 reachable states, so it basically looked at everything. That's what pays for the guarantees. DFS looked at less and got nothing guaranteed back. Both only finish at all because the closed set stops them from looping. A DFS with no closed set would

just move the same cannibal back and forth forever. So the expansion count tells you how much work happened, not how good the answer is.

## Question 2: Uniform Cost Search

Cost Model A gave a total cost of 23, over 11 moves, with 14 expansions. Cost Model B gave 17, also 11 moves, also 14 expansions. Both returned the same path BFS did.

**Does UCS pick a different sequence?** Not this time, but that's luck. All four solutions are 11 moves, and under Model A they cost 23, 25, 25, and 27. BFS doesn't care which one it gets since they're all the same length. It only landed on the cheap one because of the order we happened to generate moves in. Change that order and BFS gives back the 27 and is still totally correct about minimizing moves. UCS gives 23 no matter what. BFS got lucky, UCS didn't need luck.

**How does cost compare to path length?** Depends on the model. Under Model B, cost only depends on direction. The boat has to alternate and end on the right, so any 11 move solution is 6 trips over and 5 back, which is  $6(2) + 5(1) = 17$  every single time. Cost is just length in disguise there, so UCS and BFS do the same thing. Under Model A that falls apart, because cost depends on who's in the boat, not how many trips. Two paths of the same length cost 23 and 27. That's when you actually need UCS.

**Which model expands more?** Both were 14, same as BFS. With only 16 states there's no room for a difference. On a bigger puzzle I'd guess Model B expands fewer, since it only has two cost values so lots of paths tie and the goal pops out early. Model A has costs from 1 to 4, so the queue splits things up more and UCS has to dig further before it's sure.

**Why UCS is needed.** BFS is only optimal because of an assumption. If every step costs 1, then shortest and cheapest are the same thing. Break that and the whole argument dies, because a short path made of expensive moves can lose to a longer path made of cheap ones. UCS sorts the fringe by  $g(n)$ , the cost so far, instead of by depth. So it pops things in order of real cost and knows nothing cheaper is left when it hits the goal. It's basically Dijkstra, and BFS is just the version where every cost is 1, which is exactly why they match under Model B.

Least cost versus fewest steps really comes down to what you actually care about. Under Model A the cheapest plan is 23 and takes 11 moves, and there's another 11 move plan that costs 27. If those numbers are a rowing effort, the 23 is better and the extra 4 is wasted. But they can also fight each other. A cheapest plan will happily take extra trips if they're cheap, so you get something longer and more annoying. A shortest plan can be quick but exhausting for whoever's rowing. The cost function is where you say which one you want, and the algorithm just follows it.

## Question 3 Part A: Admissibility

All of Question 3 uses Cost Model A. It helps to give everyone a weight: 2 for a missionary, 1 for a cannibal. Then a trip costs the total weight in the boat, and a whole plan costs each person's weight times

how many times they cross. I'll write  $W$  for  $2 \cdot M_{\text{left}} + C_{\text{left}}$ , the weight still on the left, which is the same thing as  $h_1$ .

**$h_1$  is admissible.** Everyone on the left has to end up on the right, so each of them crosses at least once. Since a trip costs the sum of the weights in it, any solution has to include at least a 2 for every missionary on the left and a 1 for every cannibal. Everything else in that sum is 0 or more, so throwing those terms away only makes the number smaller. That means every solution costs at least  $W$ , which is  $h_1$ , so  $h_1$  is never bigger than the true cost. It's also 0 at the goal. Basically  $h_1$  charges you for sending everyone over once and forgets the boat has to come back, so it can only guess low.

**$h_2$  is admissible.** For any whole number  $W$  that's 0 or more,  $\text{ceil}(W/3)$  is at most  $W$ . If  $W$  is 0 both sides are 0, and if  $W$  is 1 or more then  $\text{ceil}(W/3) \leq (W+2)/3 \leq W$ , since  $W+2 \leq 3W$ . So  $h_2 \leq h_1 \leq \text{true cost}$ , and  $h_2$  is 0 at the goal too. This also shows  $h_1$  dominates  $h_2$ .  $h_2$  is really weak. At the start it says 3 when the real answer is 23, so I'd expect  $A^*$  with  $h_2$  to act almost exactly like plain UCS.

## Question 3 Part B: Results

| Heuristic | Solution Path        | Total Cost | Node Expansions | Same Cost as UCS |
|-----------|----------------------|------------|-----------------|------------------|
| $h_1$     | 11 moves, path above | 23         | 13              | yes              |
| $h_2$     | 11 move, path above  | 23         | 14              | yes              |
| $h_3$     | 11 moves, path above | 23         | 13              | yes              |

All three matched UCS at 23, which is what should happen. All three are admissible, and  $A^*$  with an admissible heuristic is optimal, so if they hadn't matched I'd have gone looking for a bug.

**Which one was more efficient?**  $h_1$  and  $h_3$  both expanded 13,  $h_2$  expanded 14, which is the same as UCS with no heuristic at all. That order makes sense from dominance.  $h_3 \geq h_1 \geq h_2$  everywhere, and a bigger admissible heuristic is always at least as good, because  $A^*$  has to expand everything with  $f$  below the optimal cost, and raising  $h$  shrinks that group.  $h_2$ 's numbers are so small that  $g$  basically decides  $f$  on its own, so  $A^*$  turns into UCS. The 14 shows that.

That said, a difference of one node isn't really a difference, and it's a little disappointing that  $h_3$  didn't beat  $h_1$  even though it's a much better estimate. That's the puzzle's fault, not the heuristic's. With 16 states there's only a node or two between a great heuristic and a useless one, so there's nowhere for the

improvement to show up. You'd need something bigger, like 5 or 6 of each with a boat that holds 3 or 4, before they'd actually separate. I'd rather report what we got than pretend the gap means something.

## Question 3 Part C: My Own Heuristic

What bugs me about  $h_1$  is that it pays to send the left bank people over once and then stops. But the boat can't row itself back. Every return trip needs somebody in it, and that person has to be brought over again later. Those are two crossings  $h_1$  never charges for, so  $h_3$  adds them.

Let  $N = M_{\text{left}} + C_{\text{left}}$  be the people still on the left, and let  $R$  be the fewest return trips still needed. Then:

**$h_3 = h_1 + 2R$** , where  $R = \max(0, N - 2)$  if the boat is on the left, and  $R = N$  if the boat is on the right.

At the start  $N$  is 6 and the boat is on the left, so  $R$  is 4 and  $h_3 = 9 + 8 = 17$ . Compare that to  $h_1$  at 9,  $h_2$  at 3, and the real answer of 23.

**Why  $R$  is that big.** This is the part we had to think about. The boat holds 2, but every trip back has to carry at least one person, so a forward trip paired with a return trip only nets you 1 person delivered, not 2. Only the very last forward trip gets to deliver 2 with nobody coming back after it.

Say there are  $F$  forward trips and  $R$  return trips. The number of people actually delivered is the people carried forward minus the people carried back, which is at most  $2F - R$ . If the boat starts on the left it ends with a forward trip, so  $F = R + 1$ , and that gives  $2F - R = F + 1$ . We need to deliver all  $N$ , so  $N \leq F + 1$ , meaning  $F \geq N - 1$  and  $R \geq N - 2$ . If the boat starts on the right it has to come back first, so  $F = R$ , and  $2F - R = R$ , which forces  $R \geq N$ .

**Why that makes  $h_3$  admissible.** Why that makes  $h_3$  admissible. Each of those forced return trips costs at least 1, since somebody has to row back and the lightest person is a cannibal at weight 1. That same person is then on the left bank and still has to end up on the right, so they cross forward again later for at least 1 more. So each forced return adds at least 2 that  $h_1$  never counted, and none of it overlaps with  $h_1$ , since  $h_1$  only counts each left bank person's first trip over. So the true cost is at least  $h_1 + 2R = h_3$ . We also compared  $h_3$  against the exact true cost for all 16 states and nothing violated it.

**Why it dominates.**  $R$  is never negative, so  $h_3 = h_1 + 2R$  is always at least  $h_1$ , and  $h_1$  is at least  $h_2$  from Part A. It's strictly bigger than  $h_1$  at 12 of the 16 states, including the start, where it's 17 against 9. It's also exact at 8 states where  $h_1$  is only exact at 4, so it's a lot closer to the truth overall. And it isn't  $\max(h_1, h_2)$ , because  $h_1$  already dominates  $h_2$  so that  $\max$  would just be  $h_1$ , and  $h_3$  beats  $h_1$  almost everywhere.

## Question 3.3 Extra Credit: Consistency

Consistent means  $h(s) \leq c + h(s')$  on every edge. A move with  $m$  missionaries and  $k$  cannibals costs  $c = 2m + k$ , which is at least 1 because the boat can't cross empty, and it's also at least  $j$ , the number of people

in the boat. Going forward drops  $W$  by exactly  $c$ , going back raises it by exactly  $c$ . All three heuristics are consistent, and the proofs are below.

**h1.** Going forward,  $W$  drops by  $c$ , so  $h1(s) = c + h1(s')$  exactly. Equality works. Going back,  $W$  goes up by  $c$ , so  $h1(s')$  is bigger than  $h1(s)$  and it holds easily. The equality going forward is kind of cool. It means  $h1$  is exactly the cost of a version of this puzzle where the boat teleports back for free.

**h2.** Going forward, the ceiling is subadditive, so  $\text{ceil}(W/3) \leq \text{ceil}((W-c)/3) + \text{ceil}(c/3)$ , and  $\text{ceil}(c/3) \leq c$  since  $c$  is a positive whole number. Going back,  $W$  only goes up and the ceiling never goes down, so  $h2(s') \geq h2(s)$ .

**h3.** Say the move carries  $j$  people, so  $j$  is 1 or 2 and  $c \geq j \geq 1$ . Going forward, the boat goes from left to right,  $W$  drops by  $c$ , and  $N$  drops by  $j$ . The  $h1$  parts cancel against the cost, so what's left is  $\max(0, N-2) \leq N-j$ . Since  $j$  is at most 2 that holds, and  $N-j$  is at least 0 because you can't move more people than are standing there. Going back, the boat goes right to left,  $W$  goes up by  $c$ , and  $N$  goes up by  $j$ . Cancelling leaves  $N \leq c + \max(0, N+j-2)$ . Since  $j$  is at least 1, that max is  $N+j-2$ , which is at least  $N-1$ , so the right side is at least  $c + N - 1$ , and that's at least  $N$  because  $c$  is at least 1. The only edge case is  $N+j < 2$ , which means  $N$  is 0 and  $j$  is 1, and then you just need  $0 \leq c$ , which is fine.

This isn't just for the extra credit either. Consistency is what makes the closed set in my  $A^*$  safe. If a heuristic were admissible but not consistent, skipping a state I'd already closed could hand me a worse path and I'd have to reopen nodes. Since all three are consistent, my code is fine as is.